Strategic Analysis: Ecommerce Revenue Optimization

Prepared by: Seiha Vat

Date: February 9, 2026

---

1. Data Summary: The "Digital Footprint"

Our dataset comprises **500 unique customer profiles**, capturing the interplay between digital engagement and annual revenue.

- **Key Variables (Predictors):**
  - Time on App: Minutes spent on the mobile platform.
  - Time on Website: Minutes spent on the desktop platform.
  - Length of Membership: Years the customer has been with the brand.
- **Target Variable:** Yearly Amount Spent ($) — our primary KPI.

---

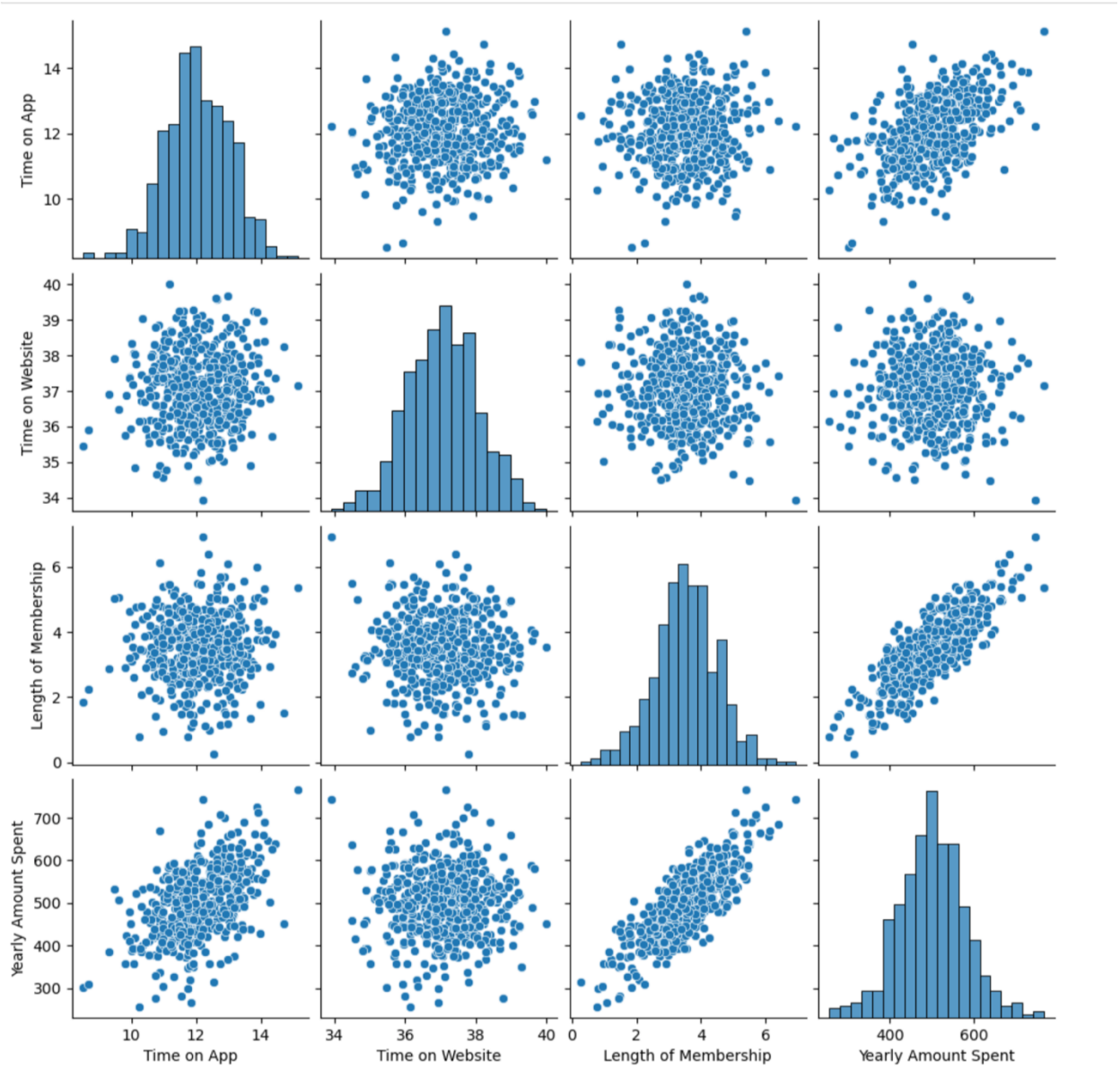
2. Objective of the Analysis

The primary goal is to **identify the high-leverage drivers of revenue** to guide the company's 2026 technical roadmap. Specifically, we must resolve a critical resource allocation dilemma: **Should we invest in a major Website overhaul or double down on Mobile App engagement?**

---

3. Exploratory Data Analysis (EDA)

Before modeling, we visualized the relationships between features to identify multicollinearity and strong predictors.

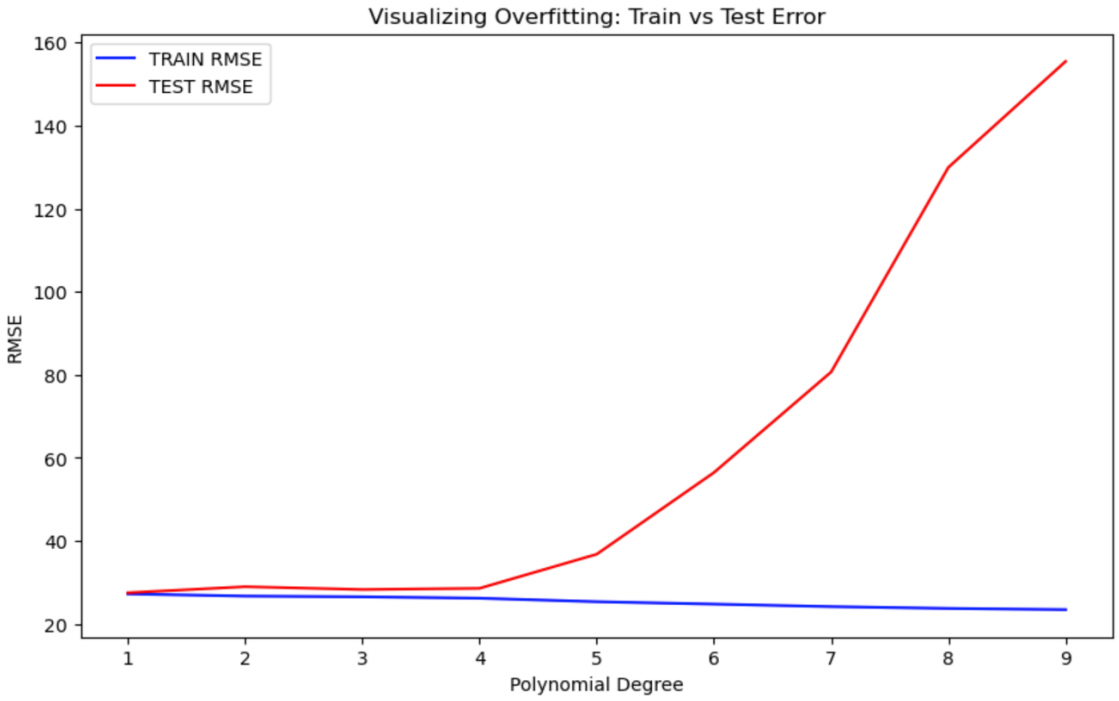


- ***Observation:*** **Length of Membership and Time on App show the clearest linear relationship with Yearly Amount Spent.**

4. The "Scientist" Phase: Investigating Complexity

Before building the final model, I investigated if a complex, non-linear relationship existed. This prevents "overfitting"—where a model learns noise rather than actual trends.

```
from sklearn.preprocessing import PolynomialFeatures

train_rmse_errors = []
test_rmse_errors = []

for d in range(1,10):
    #Transformation
    poly_converter = PolynomialFeatures(degree = d, include_bias = False)
    X_train_poly = poly_converter.fit_transform(X_train)
    X_test_poly = poly_converter.transform(X_test)

    lr_poly = LinearRegression().fit(X_train_poly, y_train)
    y_train_pred_poly = lr_poly.predict(X_train_poly)
    y_test_pred_poly = lr_poly.predict(X_test_poly)

    train_rmse_errors.append(np.sqrt(mean_squared_error(y_train, y_train_pred_poly)))
    test_rmse_errors.append(np.sqrt(mean_squared_error(y_test, y_test_pred_poly)))

plt.figure(figsize = (10,6))
plt.plot(range(1,10), train_rmse_errors, label = 'TRAIN RMSE', color = 'blue')
plt.plot(range(1,10), test_rmse_errors, label = 'TEST RMSE', color = 'red')
plt.xlabel('Polynomial Degree')
plt.ylabel('RMSE')
plt.title('Visualizing Overfitting: Train vs Test Error')
plt.legend()
plt.show()
```



Insight: Notice how the Test Error (Red) explodes after Degree 4. This confirms we must stay within a simple Linear or Degree-2 complexity.

---

5. The "Engineer" Phase: Production-Ready Workflow

To find the absolute "sweet spot," I implemented an industry-standard **Scikit-Learn Pipeline**. This ensures the data is scaled correctly and prevents "Data Leakage."

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import Ridge
from sklearn.model_selection import GridSearchCV

# 1. Define Pipeline
pipe = Pipeline([
    ('poly' , PolynomialFeatures()),
    ('scaler', StandardScaler()),
    ('ridge' , Ridge())])

# 2. Define Parameter Grid
# We limit degree based on our previous plot (1 to 4 is usually the sweet spot)
param_grid = {
    'poly__degree': [1,2,3,4],
    'ridge__alpha': [0.01, 0.1, 1, 10, 50]}

# 3. Grid Search with 5-fold Cross Validation
grid_model = GridSearchCV(pipe, param_grid, cv =5, scoring = 'neg_mean_squared_error')
grid_model.fit(X_train, y_train)

# 4. Final Best Parameters
print(f'Optimal Configuration: {grid_model.best_params_}')
```

---

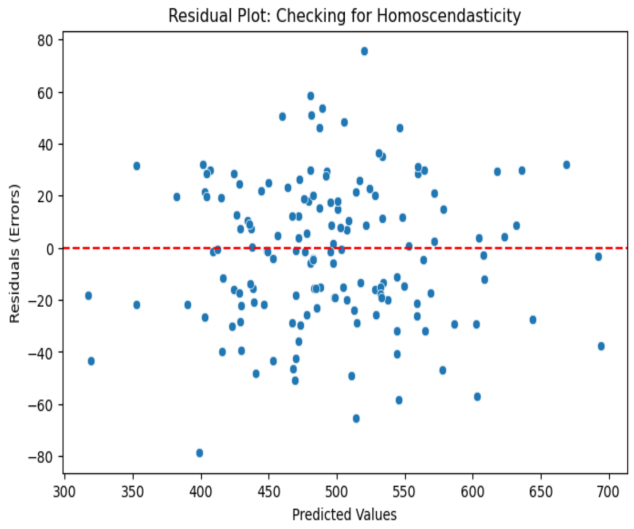
6. Model Comparison & Selection

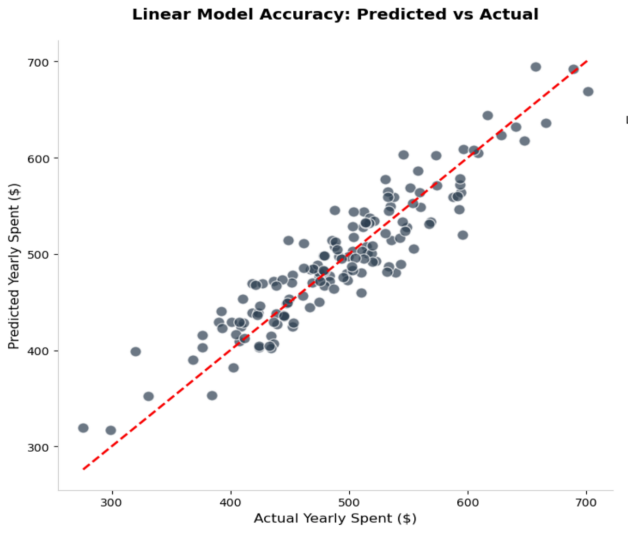
I compared three variations to determine the most stable and accurate predictor for the business.

	Model	Regression	R2 (Test)	RMSE (Test)
0		Linear	0.860355	27.546310
1		Ridge	0.850980	28.455916
2		Lasso	0.849386	28.607756

Why Linear Regression Won

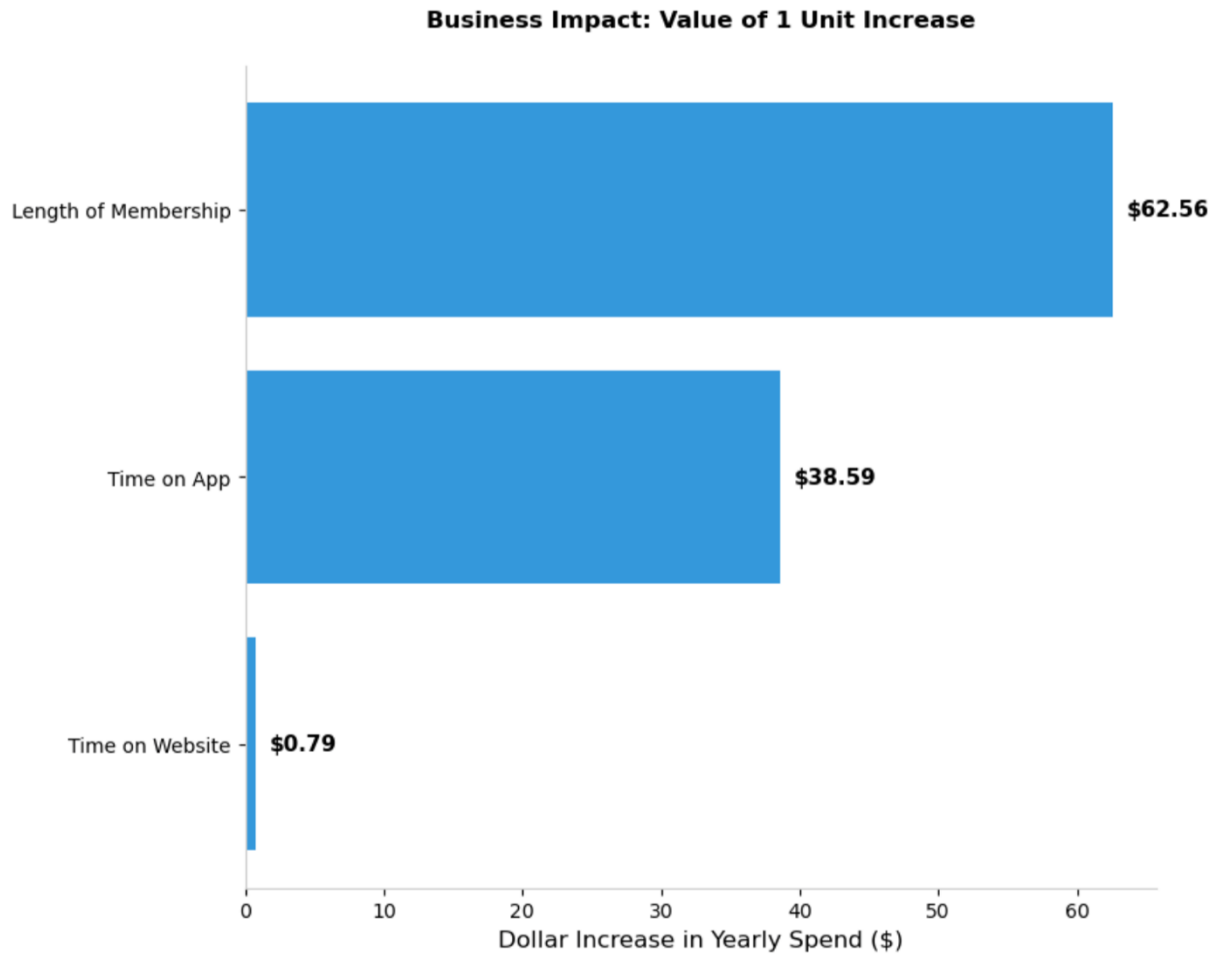
While we explored advanced **Ridge** and **Lasso** techniques via a GridSearchCV pipeline, the data demonstrated a purely linear nature. because the relationship is fundamentally linear (as seen in your EDA pairplot), the "penalty" added by Ridge or Lasso regression actually pushed the model away from the true optimal solution.. **The Simple Linear Model is the "Winner" because it is both the most accurate and the most explainable.**





7. Key Findings: Where the Money Is

Our winning model provides a mathematical roadmap for the CEO.



- **The App vs. Web Verdict:** The **Mobile App** yields approximately **$39.59** in revenue per minute, whereas the **Website** yields a negligible **$0.79**. The App is **50 times** more effective at driving spend.
- **The Loyalty Multiplier: Length of Membership** is the single greatest predictor of wealth. For every year a customer stays, their annual spend increases by roughly **$62.56**.

---

8. Limitations and Next Steps

No model is perfect. Recognizing flaws is a sign of a senior-level analyst.

- **Model Flaws (Limitations):**
  - **External Factors:** The current model ignores external variables like marketing spend, season, or competitor pricing.
  - **Linearity Assumption:** While currently linear, customer behavior may shift (plateau) at very high levels of app usage.
- **Future Improvements (Next Steps):**
  - **Churn Prediction:** We should now build a **Classification Model** to predict *which* customers are likely to cancel their membership (since membership is our biggest revenue driver).
  - **A/B Testing:** I recommend an A/B test on the Mobile App to see if specific UI changes can further increase "Time on App" now that we know its high dollar value.